

Running the CARLA simulator

In one terminal, start the CARLA simulator at a 30hz fixed time-step:

Ubuntu:

```
1 ./CarlaUE4.sh /Game/Maps/Course4 -windowed -carla-server -benchmark -fps=30
```

Windows:

```
1 CarlaUE4.exe /Game/Maps/Course4 -windowed -carla-server -benchmark -fps=30
```

Note that both the **ResX=<pixel_width>** and **ResY=<pixel_height>** arguments can be used to create a fixed size window, if you find the simulation to run too slow. See the CARLA installation guide for more details on how to use the arguments.

Running the Python client (and controller)

In another terminal, change the directory to go into the "Course4FinalProject" folder, under the "PythonClient" folder.

Run your controller, execute the following command while CARLA is open:

Ubuntu (use alternative python commands if the command below does not work, as described in the CARLA install guide):

```
1 python3 module_7.py
```

Windows (use alternative python commands if the command below does not work, as described in the CARLA install guide):

```
1 python module_7.py
```

The simulator will begin to run if the module_7.py client connects to the server properly. It will open two new feedback windows (unless live_plotting is disabled - see the **changing the live plotter refresh rate** section below for more details), one of which shows the top-down trajectory and the other which shows the controls feedback.

Changing the live plotter refresh rate

If the simulation runs slowly, you can try increasing the period at which the live plotter refreshes the plots, or disabling the live plotting altogether. Disabling the live plotting or changing its refresh rate does not affect the plot outputs at the end of the simulation.

To do this, edit the **options.cfg** file found in the "Course1FinalProject" folder for the relevant parameters. The following table below explains each option:

Parameter	Description	Value
live_plotting	Enable or disable live plotting.	true/false
live_plotting_period	Period (in seconds) which the live plot will refresh on screen. Set to "0" for an update every simulation timestep.	[seconds]

Evaluating the simulation results

This will then start the simulation and will run your developed Python client. Once you reach the end of the scenario, the Python client will output controller_output/trajectory.txt and controller_output/collision_count.txt. You should submit both of these files to Coursera for grading. You will be graded on generating a collision-free path to the goal, without deviating too greatly from the waypoints both in terms of speed and velocity. You will also need to come to a complete stop in the designated stop region before the stop sign as well.